KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

uv로 파이썬 시작하기

예전 방식과 비교하고,
자주 쓰는 명령 익히기

```
$ uv init hello-uv
  프로젝트 만들기
$ uv add requests
  패키지 넣기
$ uv run main.py
  실행하기
$ uv sync
  다른 컴퓨터에서 그대로
```

오늘의 순서

1 <>

파이썬은 어떻게 실행되나

python, 패키지

3–4쪽

2

예전 방식

venv, pip, requirements.txt

5–8쪽

3

uv는 무엇이 다른가

비교와 설치

9–12쪽

4

프로젝트 만들고 실행하기

uv init, uv run

13–17쪽

5

패키지 넣고 빼기

uv add, uv tree, uv remove

18–22쪽

6

다른 컴퓨터에서 그대로

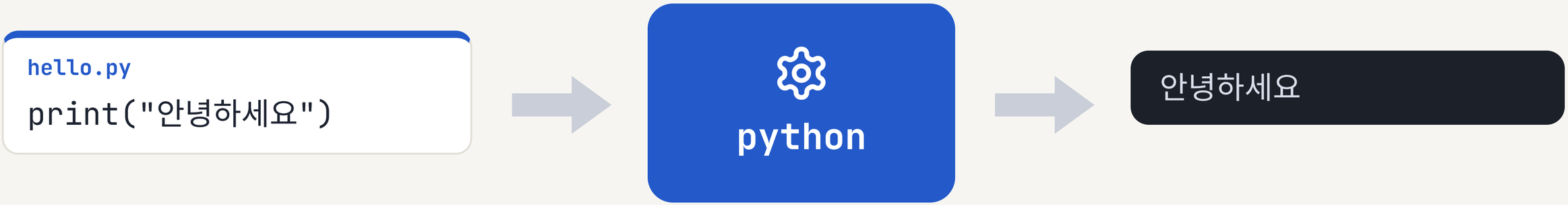
GitHub, uv sync

23–26쪽

27–29쪽은 명령 바꿔 치기, 막힐 때, 한 장 요약이에요.

1. 파이썬은 어떻게 실행되나

파이썬 코드는 python이 읽고 실행해요




```
$ python3 hello.py
안녕하세요
```

.py 파일은 글자일 뿐이에요. python 프로그램이 한 줄씩 읽어서 실행해요.

1. 파이썬은 어떻게 실행되나

남이 만든 코드 묶음을 패키지라고 해요

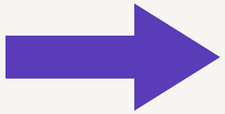
PyPI 파이썬 패키지 창고

`requests` `pandas` `numpy` `openai`

`fastapi` `matplotlib` `pytest`

`… 60만 개 넘게`

받아서 설치



내 프로젝트/

`main.py`

`requests`

`import requests` 한 줄로
인터넷에 요청을 보낼 수 있어요.

이 자료는 requests 패키지 하나로 넣고, 쓰고, 빼 봐요.

2. 예전 방식

예전에는 이 순서를 직접 챙겼어요

1	파이썬 설치	python.org에서 받아 설치	python
2	가상환경 만들기	`python3 -m venv .venv`	venv
3	가상환경 켜기	`source .venv/bin/activate`	venv
4	패키지 설치	`pip install requests`	pip
5	목록 적어 두기	`pip freeze > requirements.txt`	requirements.txt
6	실행	`python main.py`	python

2. 예전 방식

가상환경은 프로젝트마다 따로 쓰는 패키지 상자예요

컴퓨터에 설치한 파이썬 여기에 바로 넣으면 모든 프로젝트가 같이 써요

project-a/

main.py

.venv/

requests 2.34.2

project-b/

main.py

.venv/

requests 2.28.0

프로젝트끼리 패키지 버전이 달라도 부딪히지 않아요. uv도 이 .venv를 그대로 써요.

2. 예전 방식

예전 방식을 실제로 쳐 보면

```
$ python3 main.py
ModuleNotFoundError: No module named 'requests'
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip install requests
Collecting requests
  ... 중간 줄 생략 ...
Successfully installed certifi-2026.10.23 charset_normalizer-3.5.2 idna-3.20 requests-2.34.2 urllib3-2.8.0
(.venv) $ pip freeze > requirements.txt
(.venv) $ python main.py
상태 코드: 200
```

가상환경을 켜면 프롬프트 앞에 (.venv)가 붙어요. 켠 상태에서만 패키지가 보여요.

2. 예전 방식

예전 방식에서 자주 막히는 곳

켜는 걸 잊으면

deactivate 뒤에 python3 main.py

ModuleNotFoundError

목록은 손으로

pip install 뒤 pip freeze를 잊으면

requirements.txt가 옛날 그대로

빼도 남아요

pip uninstall requests 뒤에도

certifi, idna 같은 4개가 남아요

파이썬은 따로

설치와 버전 맞추기는 직접

python, python3 이름도 컴퓨터마다 달라요

3. uv는 무엇이 다른가

예전에는 일마다 도구가 달랐어요

파이썬 설치

`python.org`

가상환경

`venv, activate`

패키지 설치

`pip`

목록 적기

`requirements.txt`

네 가지를 각각 배우고, 순서도 직접 기억해야 했어요.

3. uv는 무엇이 다른가

uv 하나가 네 가지를 다 해요

UV

파이썬 설치

필요하면 알아서 받아요

가상환경

uv run이 만들고 켜요

패키지 설치

uv add

목록 적기

uv.lock에 자동으로

명령은 uv 하나로 시작해요. 가상환경을 켜고 끄는 일은 없어요.

3. uv는 무엇이 다른가

할 일마다 비교해 봐요

할 일	예전	uv
파이썬 설치	python.org에서 받아 직접 설치	**없으면 uv가 받아요**
가상환경	python3 -m venv .venv 뒤 activate	`uv run이 알아서 만들고 켜요`
패키지 넣기	pip install, 그다음 pip freeze	`uv add requests`
패키지 빼기	pip uninstall, 딸린 패키지는 남아요	`uv remove requests`
다른 컴퓨터	pip install -r requirements.txt	`uv sync`
실행	켠 상태에서 python main.py	`uv run main.py`

uv 설치하기

macOS, Linux 터미널에 붙여 넣기

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows PowerShell에 붙여 넣기

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

3 터미널을 닫았다가 새로 열어요. Windows는 Git Bash를 새로 열어요.

```
$ uv --version
uv 0.11.32 (x86_64-unknown-linux-gnu)
```

4. 프로젝트 만들고 실행하기

uv init으로 프로젝트를 만들어요

```
$ cd ~
$ uv init hello-uv
Downloading cpython-3.14.6-linux-x86_64-gnu (download) (34.3MiB)
 Downloaded cpython-3.14.6-linux-x86_64-gnu (download)
Initialized project `hello-uv` at `/home/kim/hello-uv`
$ cd hello-uv
$ ls -a
```

hello-uv/

.git	Git 저장소 (02)
.gitignore	Git에 안 올릴 목록
.python-version	쓸 파이썬 버전
README.md	프로젝트 소개 글
main.py	실행할 코드
pyproject.toml	프로젝트 설명서

컴퓨터에 파이썬이 없으면 uv가 받아서 써요. 위 두 줄이 그 표시예요.

pyproject.toml은 프로젝트 설명서예요

pyproject.toml

```
[project]
name = "hello-uv"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.14"
dependencies = []
```

◀ 프로젝트 이름

◀ 쓸 수 있는 파이썬 버전

◀ 쓰는 패키지, 아직 없어요

.python-version

```
3.14
```

main.py

```
def main():
    print("Hello from hello-uv!")


if __name__ == "__main__":
    main()
```

4. 프로젝트 만들고 실행하기

uv run으로 실행해요

```
$ uv run main.py
Using CPython 3.14.6
Creating virtual environment at: .venv
Hello from hello-uv!
```

hello-uv/

.git	Git 저장소 (02)
.gitignore	Git에 안 올릴 목록
.python-version	쓸 파이썬 버전
.venv	가상환경 (패키지 상자)
README.md	프로젝트 소개 글
main.py	실행할 코드
pyproject.toml	프로젝트 설명서
uv.lock	정확한 버전 목록

처음 한 번 .venv와 uv.lock을 만들어요. activate는 치지 않아요.

4. 프로젝트 만들고 실행하기

두 번째부터는 바로 실행돼요

```
$ uv run main.py
Hello from hello-uv!
```



예전

```
source .venv/bin/activate
python main.py
```

uv

```
uv run main.py
```

uv run은 켜고, 실행하고, 끄는 일을 한 번에 해요.

파이썬 버전은 .python-version이 정해요

```
$ cd ~
$ uv init --python 3.13 hello-313
Initialized project `hello-313` at `/home/kim/hello-313`
$ cd hello-313
$ cat .python-version
3.13
$ uv run python --version
Downloading cpython-3.13.14-linux-x86_64-gnu (download) (33.2MiB)
 Downloaded cpython-3.13.14-linux-x86_64-gnu (download)
Using CPython 3.13.14
Creating virtual environment at: .venv
Python 3.13.14
```

버전을 정해서 만들려면 --python을 붙여요. 없는 버전은 uv가 받아요.

5. 패키지 넣고 빼기

uv add로 패키지를 넣어요

```
$ uv add requests
Resolved 6 packages in 216ms
Prepared 5 packages in 83ms
Installed 5 packages in 2ms
 + certifi==2026.10.23
 + charset-normalizer==3.5.2
 + idna==3.20
 + requests==2.34.2
 + urllib3==2.8.0
```

pyproject.toml 내가 고른 패키지

```
dependencies = [
    "requests>=2.34.2",
]
```

uv.lock 따라온 것까지 정확한 버전

requests와 따라온 4개

certifi 2026.10.23, idna 3.20 …

.venv 실제로 설치된 패키지

패키지 5개 설치

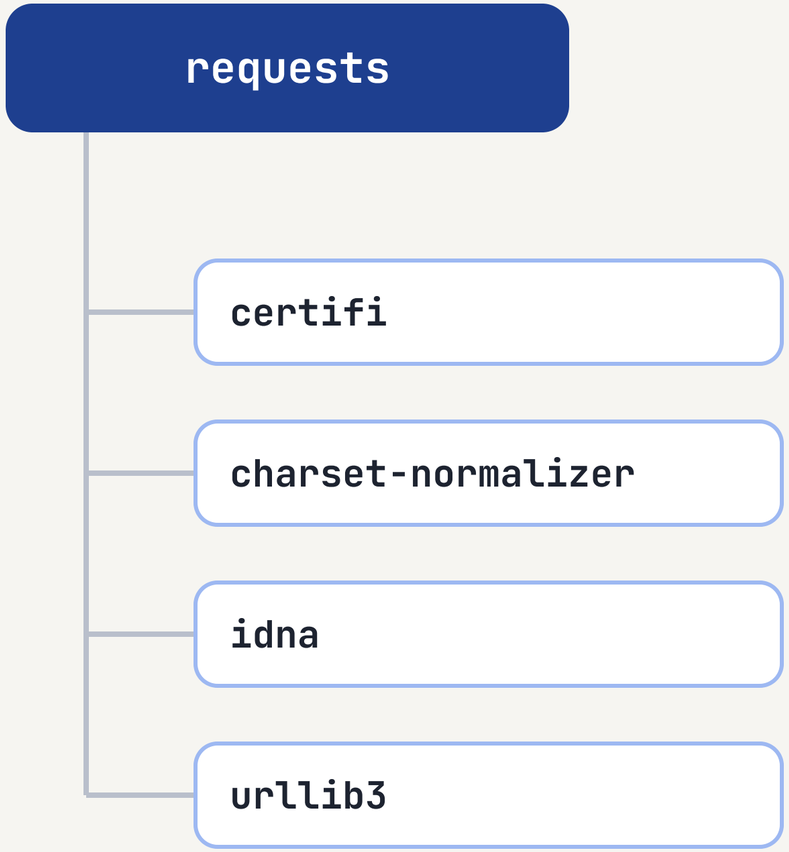
uv run으로 실행하면 바로 써요

명령 하나로 세 곳이 함께 바뀌어요.

5. 패키지 넣고 빼기

하나를 넣었는데 다섯 개가 들어왔어요

```
$ uv tree
Resolved 6 packages in 1ms
hello-uv v0.1.0
└─── requests v2.34.2
     ├─── certifi v2026.10.23
     ├─── charset-normalizer v3.5.2
     ├─── idna v3.20
     └─── urllib3 v2.8.0
```



requests가 쓰는 패키지까지 uv가 함께 넣어요. 뺄 때도 함께 빠져요.

5. 패키지 넣고 빼기

넣은 패키지를 코드에서 써요

main.py

```
import requests


def main():
    r = requests.get("https://api.github.com")
    print("상태 코드:", r.status_code)


if __name__ == "__main__":
    main()
```

```
$ uv run main.py
상태 코드: 200
```

200

요청이 잘 됐다는 뜻이에요.
GitHub 서버가 대답했어요.

VS Code에서 main.py를 이렇게 고쳐 저장하고 uv run으로 실행해요.

5. 패키지 넣고 빼기

uv remove로 패키지를 빼요

```
$ uv remove requests
Resolved 1 package in 3ms
Uninstalled 5 packages in 5ms
 - certifi==2026.10.23
 - charset-normalizer==3.5.2
 - idna==3.20
 - requests==2.34.2
 - urllib3==2.8.0
$ uv run main.py
ModuleNotFoundError: No module named 'requests'
```

pyproject.toml 내가 고른 패키지

dependencies = []

uv.lock 따라온 것까지 정확한 버전

프로젝트만 남아요

따라왔던 4개도 지워요

.venv 실제로 설치된 패키지

패키지 5개 삭제

딸린 패키지까지 같이 빠져요

뺄 때도 세 곳이 함께 바뀌어요. 예전 pip uninstall과 달리 하나도 남지 않아요.

5. 패키지 넣고 빼기

다시 넣으면 바로 들어가요

```
$ uv add requests
Resolved 6 packages in 4ms
Installed 5 packages in 2ms
 + certifi==2026.10.23
 + charset-normalizer==3.5.2
 + idna==3.20
 + requests==2.34.2
 + urllib3==2.8.0
$ uv run main.py
상태 코드: 200
```

uv 캐시

한 번 받은 패키지를 컴퓨터에 보관해요.
다른 프로젝트에서도 다시 받지 않아요.

처음 넣을 때 있던 줄

`Prepared 5 packages in 83ms`

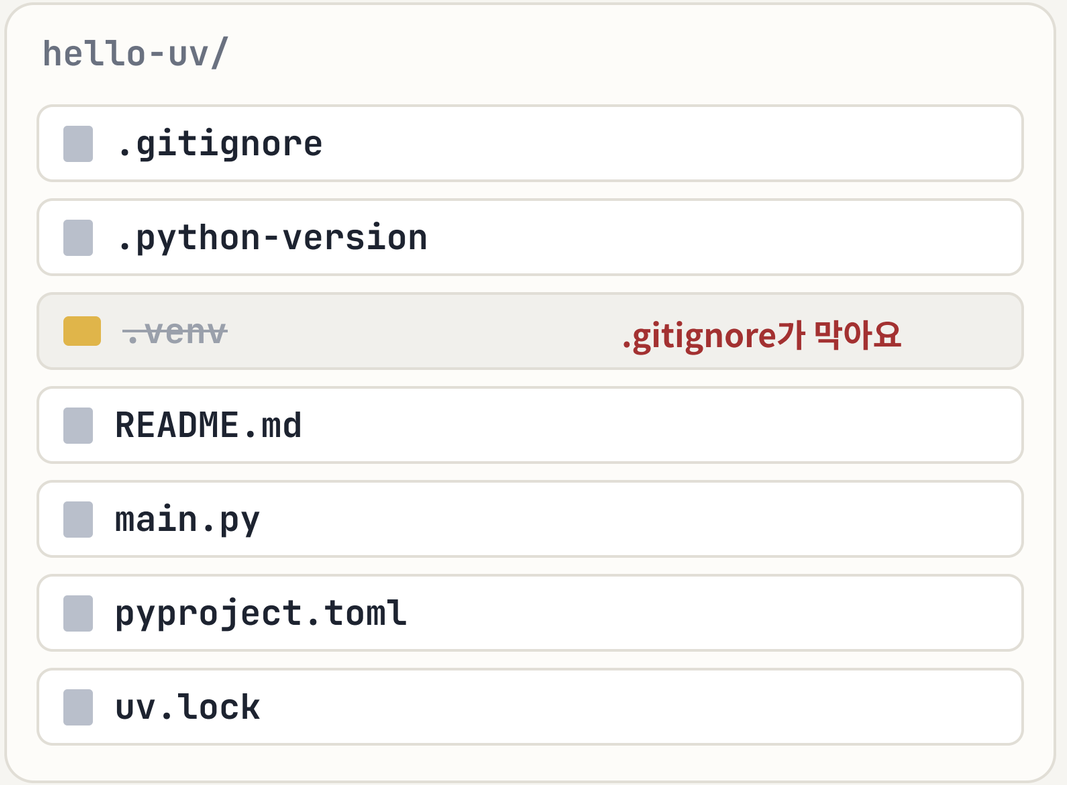
이번엔 받을 게 없어서 이 줄이 없어요.

uv가 빠른 이유 중 하나예요. 패키지 다섯 개가 0.002초 만에 들어갔어요.

6. 다른 컴퓨터에서 그대로

GitHub에는 무엇을 올리나요

```
$ git status --short
?? .gitignore
?? .python-version
?? README.md
?? main.py
?? pyproject.toml
?? uv.lock
$ git add .
$ git commit -m "feat: hello-uv 프로젝트 만들기"
[main (root-commit) 749f4de] feat: hello-uv 프로젝트 만들기
```



.venv는 올리지 않아요. pyproject.toml과 uv.lock만 있으면 똑같이 다시 만들어요.

6. 다른 컴퓨터에서 그대로

lee 컴퓨터에서는 uv sync 한 번이면 돼요



uv.lock에 적힌 버전 그대로 설치돼서 kim과 lee의 환경이 같아요.

6. 다른 컴퓨터에서 그대로

lee 컴퓨터에서 실제로 쳐 보면

```
$ git clone https://github.com/kim/hello-uv.git
Cloning into 'hello-uv'...
$ cd hello-uv
$ uv sync
Downloading cpython-3.14.6-linux-x86_64-gnu (download) (34.3MiB)
 Downloaded cpython-3.14.6-linux-x86_64-gnu (download)
Using CPython 3.14.6
Creating virtual environment at: .venv
Resolved 6 packages in 1ms
Prepared 5 packages in 128ms
Installed 5 packages in 5ms
 + certifi==2026.10.23  + charset-normalizer==3.5.2  + idna==3.20  …
$ uv run main.py
상태 코드: 200
```

6. 다른 컴퓨터에서 그대로

uv sync는 이럴 때 쳐요

1

clone한 직후

```
git clone 주소
```

```
cd 폴더
```

```
uv sync
```

2

pull로 uv.lock이 바뀐 뒤

```
git pull origin main
```

```
uv sync
```

3

uv run도 먼저 맞춰요

```
uv run main.py
```

실행 전에 sync를 알아서 해요

팀원이 패키지를 넣고 push했으면, pull 뒤 uv sync로 내 .venv를 맞춰요.

정리

예전 명령은 이렇게 바뀌었어요

`python.org에서 설치`	→	없으면 uv가 받아요
`python3 -m venv .venv`	→	uv run, uv sync가 만들어요
`source .venv/bin/activate`	→	필요 없어요
`pip install requests`	→	`uv add requests`
`pip uninstall requests`	→	`uv remove requests`
`pip install -r requirements.txt`	→	`uv sync`
`python main.py`	→	`uv run main.py`

정리

이런 메시지가 나와요

1

`bash: uv: command not found`

설치 뒤 터미널을 새로 열지 않았어요

터미널 닫고 다시 열기

2

`error: No `pyproject.toml` found in current directory or any parent directory`

프로젝트 폴더 밖에서 쳤어요

```
cd hello-uv
```

3

`ModuleNotFoundError: No module named 'requests'`

패키지를 안 넣었거나 python3로 실행했어요

```
uv add requests
uv run main.py
```

4

`error: Project is already initialized in `...``

이미 만든 프로젝트 안에서 uv init을 쳤어요

uv init은 처음 한 번만

한 장 요약

`uv init 이름`

프로젝트 폴더 만들기

`uv run main.py`

가상환경에서 실행

`uv add 패키지`

패키지 넣기

`uv remove 패키지`

패키지 빼기

`uv sync`

uv.lock대로 맞추기

`uv tree`

패키지 관계 보기

GitHub에 올려요 `pyproject.toml uv.lock .python-version`

올리지 않아요 `.venv` uv init이 .gitignore에 적어 둬요